

BackupSage roadmap

BackupSage is becoming offline archive intelligence and recovery tooling: search, duplicate analysis, source diffs, copy coverage, integrity checks, reviewable plans, and safe recovery. Release gates, not dates, determine when each level of risk can ship.

Permanent safety invariant

BackupSage never rewrites or deletes from an archive. Archive extraction is copy-only. Any directory mutation must use a versioned, persisted plan; revalidate its preconditions; avoid clobbering existing files; and leave a durable execution journal.

Mutating actions wait until v1.2 because v1.0.1 and v1.0.2 first establish safe output paths and correctness, while v1.1 defines immutable, reviewable plans. `organize` is especially unsafe to introduce early: moving a sole copy changes its pathname and partial runs are not naturally idempotent. Deduplication also needs quarantine and replica-floor protections. Neither action should invent the shared execution contract while operating on user data.

Release sequence

v1.0.1 safety → v1.0.2 correctness → v1.1 intelligence and plans → v1.2 controlled actions → v1.3 media and scale → v2.0 local web → v2.1 remote sources

v1.0.1 — Safety baseline — shipped 2026-07-23 (tag [v1.0.1](#))

Outcome: the current read-only promise is true at every output path.

Depends on: no earlier release; blocks every later milestone.

Issues:

- [x] Protect index destinations and preserve the last-good index (#3)
- [x] Make dedup report writes no-clobber and alias-aware (#36)
- [x] Validate master catalog identity before read-write open (#37)
- [x] Sanitize untrusted text and preserve lossless paths (#5)
- [x] Remediate current Rust advisories and publish v1.0.1 (#38)

Exit gate: destructive-path regressions preserve protected fixture digests and identities; interrupted indexing preserves the previous completed index; terminal and JSON path fixtures are safe and lossless; and tests, Clippy, formatting, and `cargo audit` are clean or have every remaining warning explicitly documented. No v1.0 tag ships from the current unsafe baseline.

Gate met. The shared output-safety boundary is `src/outpath.rs` and the terminal sanitizer is `src/textsafe.rs`; `tests/safety.rs` holds the destructive-path attack matrix. 95 tests, Clippy -D warnings, formatting and `cargo audit` all clean; the two remaining unmaintained-crate warnings are dispositioned in [docs/SECURITY.md](#). Rationale is recorded in [ADR 0001](#) and [ADR 0002](#).

v1.0.2 — Correctness and debt

Outcome: known specification gaps close before new workflows are added.

Depends on: v1.0.1 safety baseline.

Issues:

- [] Add a sparse-tar differential end-to-end corpus
- [x] Make near-duplicate groups keeper-safe (#9)
- [] Add explicit content indexing modes
- [] Close the v1 debt register and release-contract gaps

Exit gate: sparse behavior has end-to-end evidence; every automatic future deletion candidate is directly safe relative to its keeper; privacy behavior is unambiguous in index mode and schema metadata; and public JSON fixtures plus an executable release checklist cover the remaining contract work.

v1.1 — Read-only intelligence and immutable plans

Outcome: users can understand change, coverage, integrity, and proposed actions without changing data.

Depends on: v1.0.2 correctness and debt closure; its plan contract blocks the v1.2 executor.

Issues:

- [] Freeze the immutable action-plan contract
- [] Add source snapshot diff
- [] Add coverage and replica-health reporting
- [] Add read-only integrity checks
- [] Generate organize, extract, and dedup plans without applying them

Exit gate: commands have stable JSON fixtures and honest incomplete-data states; plans are deterministic; a regular persisted plan is reviewable before any later apply command can accept it; and no v1.1 command moves, deletes, overwrites, or extracts user data.

v1.2 — Controlled directory actions

Outcome: reviewed plans can execute with recovery paths while archives stay immutable.

Depends on: v1.1's immutable action-plan contract.

Issues:

- [] Build the transactional plan executor
- [] Execute directory organize plans
- [] Execute copy-only archive extraction plans
- [] Apply directory dedup plans through quarantine
- [] Add keep policies and replica protections

Exit gate: kill-and-resume tests prove action-ID idempotence; changed inputs, stale plans, aliases, and conflicts fail closed; organize rejects cross-device plans, extraction proves safe destination-local no-clobber promotion, and quarantined files can be restored before a separately reviewed purge. Near duplicates are never automatically selected, and no code path opens an archive for write or delete.

v1.3 — Media formats and scale

Outcome: optional format support and larger catalogs arrive without weakening determinism or default portability.

Depends on: stable action invariants from v1.2; research may start earlier but cannot ship first.

Issues:

- [] Add opt-in HEIC perceptual hashing
- [] Add deterministic RAW preview hashing
- [] Support thresholds above three with benchmarked MIH
- [] Add incremental directory re-index
- [] Parallelize federated search deterministically
- [] Add video creation metadata before video fingerprints

Exit gate: HEIC and RAW remain opt-in with adversarial/decode-limit coverage and recorded capabilities; the MIH issue first publishes target corpus sizes and an acceptable recall, latency, memory, and candidate-volume envelope, then proves threshold expansion against a brute-force oracle without bucket-cap recall loss; incremental results converge after forced full reconciliation; federated search preserves serial ordering within documented resource bounds; and video timestamp provenance is visible. Video fingerprinting does not ship without its own design and benchmarks.

v2.0 — Local web UI

Outcome: a loopback-first web application exposes the same core behavior without duplicating indexing or analysis.

Depends on: local core behavior and action invariants; the workspace split happens when this second frontend needs it.

Issues:

- [] Split the workspace into core, CLI, and web crates
- [] Publish /api/v1 contracts
- [] Add a persistent job service
- [] Build the secure loopback-first Axum UI
- [] Add web end-to-end, packaging, and upgrade tests

Exit gate: CLI and web fixtures agree for every shared operation; schema version negotiation is tested; jobs survive restart with cancellation and progress; default binding is loopback and LAN mode proves authentication, origin, CSRF, and CORS controls; and packaged installs pass the end-to-end recovery workflow.

v2.1 — Remote sources

Outcome: S3 and SSH sources behave honestly under latency, change, and partial failure.

Depends on: v2.0's proven local core, API, and job boundaries.

Issues:

- [] Define capability-based source interfaces
- [] Add version-pinned S3 and SSH adapters
- [] Add verified spooling, caching, extraction, and failure tests

Exit gate: capability negotiation prevents unsupported range or retry assumptions; each job pins a stable remote version or reports inconclusive; S3 and SSH fixtures match local indexes and BLAKE3 results without treating ETags as content hashes; and disconnect, version-change, partial-spool, cache-corruption, and resume tests fail closed without modifying a remote source or prior good cache.

Dependency gates

1. v1.0.1 blocks every other milestone.
2. v1.0.2 blocks action-plan generation and execution.
3. The v1.1 plan contract blocks the v1.2 executor.
4. v1.3 optimizations ship only after action invariants are stable.
5. The workspace split occurs in v2.0, when the second frontend exists.
6. Remote adapters ship only after the local core, API, and job boundaries are proven.

The [future-roadmap design](#) contains the detailed research and design rationale behind these milestones and gates.